December 13, 2025

[DB_PP_2025]

Q1: Steal / No-Steal Policy and Crash Recovery

A database system manages buffer pages using the **steal/no-steal policy**. During transaction **T1**, several pages are updated in memory. Before T1 commits, one updated page is written to disk. Soon after, the system crashes and T1's log records are incomplete.

(a) Identification of Policy

The database system follows a **steal policy**.

Justification: In a steal policy, the buffer manager is allowed to write modified (dirty) pages of an **uncommitted transaction** to disk before the transaction commits. Since an updated page of T1 was written to disk before commit, the system must be using a steal policy.

(b) Impact on Recovery Process

Because the system crashed before Transaction T1 committed and its log records are incomplete:

- The disk may contain **uncommitted (dirty) data** written by T1.
- During recovery, T1 must be **undone (rolled back)** to restore database consistency.

Recovery Steps:

1. The recovery system scans the log to identify uncommitted transactions.
2. Transaction T1 is detected as incomplete.
3. An **UNDO operation** is performed using the log to reverse all updates made by T1.
4. The database is restored to a consistent state as if T1 never occurred.

(c) Advantage of Steal Policy

The main advantage of the steal policy is:

- **Efficient buffer management**: The system can free buffer space by writing dirty pages to disk, even if the transaction has not yet committed.
- This allows better utilization of memory and supports higher concurrency.

Conclusion

The system uses a steal policy, which improves memory efficiency but requires UNDO operations during recovery to handle uncommitted transactions and ensure database consistency.

Q2: Deadlock in Concurrent Transactions

Transactions:

- **T1**: Transfers $300 from Account A to Account B
- **T2**: Transfers $200 from Account B to Account A

How Deadlock Occurs

Assume the system uses **Two-Phase Locking (2PL)** and **exclusive locks** for writing.

Step-by-Step Execution

Step	Transaction	Operation
1	T1	Lock Account A (exclusive)
2	T2	Lock Account B (exclusive)
3	T1	Requests lock on Account B (waits)
4	T2	Requests lock on Account A (waits)

Deadlock Condition

- T1 holds lock on Account A and waits for Account B.
- T2 holds lock on Account B and waits for Account A.

This creates a **circular wait condition**, where neither transaction can proceed. Hence, a **deadlock** occurs.

Deadlock Prevention and Resolution

Solution 1: Lock Ordering (Deadlock Prevention)

A fixed order is defined for acquiring locks.
Rule: Always lock Account A before Account B.

- T1 locks A then B
- T2 locks A then B

This eliminates circular wait and prevents deadlock.

Solution 2: Deadlock Detection and Recovery

- The system periodically checks for cycles in a **Wait-For Graph**.
- If a cycle is found, one transaction (e.g., T2) is aborted.
- The aborted transaction releases its locks, allowing the other to continue.

Solution 3: Timeout-Based Prevention

- If a transaction waits longer than a specified time, it is rolled back.
- This releases locks and breaks the deadlock.

Conclusion

Deadlock occurs due to circular waiting caused by conflicting lock requests. Using techniques such as **lock ordering**, **deadlock detection**, or **timeouts** can effectively prevent or resolve deadlocks in concurrent transactions.

Q3: ACID Properties in an Online Shopping Platform

During a sale event, multiple users interact with an online shopping platform simultaneously. The **ACID properties** ensure correct and reliable transaction processing under such conditions.

(a) User A Adds an Item to Cart but System Crashes

Related ACID Property: Atomicity
Atomicity ensures that a transaction is treated as an **all-or-nothing** operation.

- User A's action of adding an item to the cart is a transaction.

- Since the system crashes before completion, the transaction does not commit.
- Atomicity guarantees that **partial updates are not saved**.
- After recovery, the item will **not appear** in User A's cart.

Thus, the system maintains correctness by rolling back the incomplete transaction.

(b) User B Updates Inventory, User C Views Inventory Before Commit

Related ACID Property: Isolation

Isolation ensures that **concurrent transactions do not interfere** with each other.

- User B places an order that updates inventory.
- Before User B commits, User C views the inventory.
- Isolation prevents User C from seeing **uncommitted (dirty) data**.
- User C sees the **old, consistent inventory value**.

This avoids problems such as **dirty reads** and ensures transaction correctness.

(c) User D Makes Payment but Payment Status Is Not Saved

Related ACID Property: Durability

Durability ensures that once a transaction commits, its effects **persist even after failures**.

- User D's payment is successfully processed by the bank.
- Due to system failure, payment status is not saved in the database.
- Durability requires that committed payment details be **logged and stored permanently**.
- During recovery, the system uses logs to **restore the payment status**.

This ensures no loss of confirmed transactions.

Consistency Across All Scenarios

Consistency ensures that the database moves from one valid state to another.

- Inventory counts remain correct.
- Cart data reflects only valid transactions.
- Payment and order records stay synchronized.

All integrity constraints are preserved before and after transactions.

Conclusion

The ACID properties work together to ensure:

- **Atomicity**: No partial transactions
- **Consistency**: Valid database states
- **Isolation**: Safe concurrent execution
- **Durability**: Permanent committed data

Thus, ACID guarantees reliable transaction processing even during crashes and high concurrency.

Q4: Timestamp Ordering Protocol

Timestamp Ordering Protocol (TO) is a concurrency control technique that ensures serializability by ordering transactions based on their timestamps. Each transaction is assigned a unique timestamp when it starts, and all conflicting operations must follow this timestamp order.

Basic Idea

- Older transactions (smaller timestamps) must be executed before newer transactions.
- Each data item maintains:
  - **Read Timestamp (RTS)**: Largest timestamp of any transaction that read the item.
  - **Write Timestamp (WTS)**: Largest timestamp of any transaction that wrote the item.

Rules of Basic Timestamp Ordering

Let transaction T_i have timestamp $TS(T_i)$ and data item X:

- **Read(X)** by T_i is allowed if:

$$TS(T_i) \geq WTS(X)$$

  Otherwise, T_i is aborted.

- **Write(X)** by T_i is allowed if:

$$TS(T_i) \geq RTS(X) \quad \text{and} \quad TS(T_i) \geq WTS(X)$$

  Otherwise, T_i is aborted.

Challenges in Basic Timestamp Ordering

1. High Abort Rate

- If a transaction violates timestamp order, it is immediately aborted.
- This can cause frequent rollbacks, especially for long transactions.

2. Cascading Rollbacks

- A transaction may read data written by another transaction that later aborts.
- This forces dependent transactions to abort as well.

3. No Guarantee of Recoverability

- Transactions may commit in an order that causes recovery issues.

Example of Problem in Basic Timestamp Ordering

Assume:

$$TS(T_1) = 10, \quad TS(T_2) = 20$$

- T_2 writes data item X.
- T_1 later tries to write X.

Since $TS(T_1) < WTS(X)$, transaction T_1 is aborted. If T_2 aborts later, the system faces cascading rollbacks.

Strict Timestamp Ordering Protocol

Strict Timestamp Ordering (Strict TO) is an enhanced version of timestamp ordering that prevents cascading rollbacks and ensures strict schedules.

Key Rule

- A transaction is allowed to read or write a data item **only after the transaction that last wrote the item has committed**.

How Strict TO Addresses the Challenges

- Prevents reading uncommitted data.
- Eliminates cascading rollbacks.
- Ensures recoverability.
- Maintains serializability.

Example of Strict Timestamp Ordering

Assume:

$$TS(T_1) = 10, \quad TS(T_2) = 20$$

- T_2 writes X but does not commit.
- T_1 is not allowed to read or write X until T_2 commits.

This guarantees that no transaction accesses uncommitted data, ensuring safe recovery.

Conclusion

Basic Timestamp Ordering ensures serializability but suffers from frequent aborts and cascading rollbacks. Strict Timestamp Ordering improves reliability by delaying access to uncommitted data, thus providing a safer and more robust concurrency control mechanism.

Q5: Classification of Transaction Schedules (Fully Detailed Solution)

Marks: 10 Estimated Time: 30 minutes

Key Definitions (Used in All Parts)

- **Recoverable:** If T_j reads a value written by T_i, then T_j must commit only after T_i commits.
- **Non-recoverable:** A transaction commits after reading uncommitted data from another transaction that later aborts.
- **Cascading Rollback:** A transaction reads dirty data and must abort when the writer aborts.
- **Cascadeless:** Transactions read only committed data.
- **Strict:** No transaction can read or write a data item until the last writer commits or aborts.
- **Conflict Serializable:** The precedence graph contains no cycle.

(a) $R_1(X); R_2(Y); W_1(Y); C_1; A_2$

Working: T_2 reads Y before T_1 writes it. Hence no dirty read occurs. T_2 aborts independently.

Answer:

- Recoverable (no dependency violation)
- Cascadeless (no dirty read)
- Strict (no read/write conflict on uncommitted data)
- Conflict Serializable (equivalent to $T_1 \rightarrow T_2$)

(b) $R_1(X); W_1(X); R_2(X); W_2(X); C_2; C_1$

Working: T_2 reads X written by T_1 before T_1 commits and commits first.

Answer:

- Non-recoverable (commit order violated)
- Cascading Rollback Possible
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable ($T_1 \rightarrow T_2$)

(c) $R_1(X); W_1(X); R_2(X); W_2(X); A_1; A_2$

Working: T_2 reads dirty data from T_1. When T_1 aborts, T_2 must also abort.
Answer:

- Recoverable (no commit violation)
- Cascading Rollback
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable

(d) $W_1(X); R_2(X); A_1; W_2(X); C_2$

Working: T_2 reads dirty data from T_1. After T_1 aborts, the read is invalid.
Answer:

- Recoverable
- Cascading Rollback
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable

(e) $R_1(X); W_1(X); R_2(X); C_1; W_2(X); C_2$

Working: T_2 reads X written by T_1 but commits only after T_1 commits.
Answer:

- Recoverable
- Cascading Rollback Possible
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable

(f) $W_1(X); R_2(X); W_2(X); A_2; C_1$

Working: T_2 reads dirty data but aborts itself. No committed transaction depends on dirty data.

Answer:

- Recoverable
- Cascading Rollback
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable

(g) $W_1(X); W_2(Y); R_2(X); R_1(Y); W_1(Y); C_1; C_2$

Working: T_2 reads X from T_1 and T_1 reads Y from T_2. Mutual dependency forms a cycle.

Answer:

- Recoverable
- Cascading Rollback
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable $\times$(cycle exists)

(h) $R_1(X); W_1(X); R_2(X); A_1; W_2(X); C_2$

Working: T_2 reads dirty data from T_1. When T_1 aborts, cascading rollback is required.

Answer:

- Recoverable
- Cascading Rollback
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable

(i) $W_1(X); W_2(Y); W_1(Y); C_1; R_2(X); C_2$

Working: T_2 reads X only after T_1 commits. No dirty read occurs.

Answer:

- Recoverable
- Cascadeless
- Strict
- Conflict Serializable

(j) $W_1(X); W_2(Y); R_1(Y); R_2(X); A_1; A_2$

Working: Both transactions read dirty data written by each other. When one aborts, the other must also abort.

Answer:

- Recoverable
- Cascading Rollback
- Cascadeless $\times$
- Strict $\times$
- Conflict Serializable $\times$(cycle exists)

Q6: Relational Algebra and SQL Queries

Marks: 10
Estimated Time: 30 minutes

Given Schema

- **Student**(<u>StudentId</u>, StudentName)
- **Faculty**(<u>FacultyId</u>, ProfName)
- **Course**(<u>CourseId</u>, CourseName)
- **Qualified**(FacultyId, CourseId, DateQualified)
- **Section**(<u>SectionNo</u>, Semester, CourseId)
- **Registration**(SectionNo, StudentId, Semester)

Foreign Keys:

- Qualified.FacultyId $\rightarrow$ Faculty.FacultyId

- Qualified.CourseId → Course.CourseId
- Section.CourseId → Course.CourseId
- Registration.SectionNo → Section.SectionNo
- Registration.StudentId → Student.StudentId

(a) Display all courses for which Professor Miley has been qualified

Relational Algebra:

$$\pi_{CourseName}\big(Course \bowtie Qualified \bowtie \sigma_{ProfName='Miley'}(Faculty)\big)$$

SQL Query:

```
SELECT C.CourseName
FROM Course C
JOIN Qualified Q ON C.CourseId = Q.CourseId
JOIN Faculty F ON Q.FacultyId = F.FacultyId
WHERE F.ProfName = 'Miley';
```

(b) Which instructors are qualified to teach CT220?

Relational Algebra:

$$\pi_{ProfName}\big(Faculty \bowtie Qualified \bowtie \sigma_{CourseId='CT220'}(Course)\big)$$

SQL Query:

```
SELECT DISTINCT F.ProfName
FROM Faculty F
JOIN Qualified Q ON F.FacultyId = Q.FacultyId
WHERE Q.CourseId = 'CT220';
```

(c) Is any instructor qualified to teach CT220 and not qualified to teach CS208?

Relational Algebra:

$$\pi_{FacultyId}(\sigma_{CourseId='CT220'}(Qualified)) - \pi_{FacultyId}(\sigma_{CourseId='CS208'}(Qualified))$$

SQL Query:

```
SELECT FacultyId
FROM Qualified
WHERE CourseId = 'CT220'
AND FacultyId NOT IN (
    SELECT FacultyId
    FROM Qualified
    WHERE CourseId = 'CS208'
);
```

(d) How many students are enrolled in section 1345 during semester I-2012?

Relational Algebra:

$$|\sigma_{SectionNo=1345 \wedge Semester='I-2012'}(Registration)|$$

SQL Query:

```
SELECT COUNT(*) AS TotalStudents
FROM Registration
WHERE SectionNo = 1345
AND Semester = 'I-2012';
```

(e) How many students are enrolled in CT220 during semester I-2012?

Relational Algebra:

$$|\sigma_{CourseId='CT220' \wedge Semester='I-2012'}(Registration \bowtie Section)|$$

SQL Query:

```
SELECT COUNT(*) AS TotalStudents
FROM Registration R
JOIN Section S ON R.SectionNo = S.SectionNo
WHERE S.CourseId = 'CT220'
AND R.Semester = 'I-2012';
```

Q7: Hospital Management System — Normalization

Marks: 5
Estimated Time: 15 minutes

Given Table: Appointments Record

Each appointment has a unique **ApptNo**. The table stores patient, doctor, and appointment details.

(a) Conversion to First Normal Form (1NF)

Rule of 1NF:

- All attributes must contain atomic (indivisible) values.
- No repeating groups or multivalued attributes.

Issue in Given Table:

- Patient name is not atomic (Fname, Lname).

1NF Relation:

$$Appointment\_1NF(ApptNo, ApptDate, ApptTime, ApptType, PatientID, PatientFname, PatientLname, DoctorID, DoctorName, ContactNo)$$

(b) Conversion to Second Normal Form (2NF)

Primary Key: ApptNo (single attribute)

Observation: No partial dependencies exist, so table is already in 2NF.

Functional Dependencies (FDs):

- $ApptNo \rightarrow$ ApptDate, ApptTime, ApptType, PatientID, DoctorID
- $PatientID \rightarrow$ PatientFname, PatientLname
- $DoctorID \rightarrow$ DoctorName, ContactNo

(c) Conversion to Third Normal Form (3NF)

Rule of 3NF: No non-key attribute should depend transitively on the primary key.

Transitive Dependencies Identified:

$$ApptNo \rightarrow PatientID \rightarrow (PatientFname, PatientLname)$$

$$ApptNo \rightarrow DoctorID \rightarrow (DoctorName, ContactNo)$$

Resolution: Separate patient and doctor attributes into new relations.

Relations after removing transitive dependencies:

- Appointment(ApptNo, ApptDate, ApptTime, ApptType, PatientID, DoctorID)
- Patient(PatientID, PatientFname, PatientLname)
- Doctor(DoctorID, DoctorName, ContactNo)

(d) Final Schema in Third Normal Form (3NF)

- **Appointment**(ApptNo, ApptDate, ApptTime, ApptType, PatientID, DoctorID)
- **Patient**(PatientID, PatientFname, PatientLname)
- **Doctor**(DoctorID, DoctorName, ContactNo)

Foreign Keys:

- Appointment.PatientID → Patient.PatientID
- Appointment.DoctorID → Doctor.DoctorID

Q8: E-commerce Platform — ER Diagram

Marks: 5
Estimated Time: 15 minutes

Entities, Attributes, and Primary Keys

- **Customer**(CustomerID, Name, Email, Phone)
- **Address**(AddressID, Street, City, State, Zip, CustomerID)
- **PaymentMethod**(PaymentID, Type, CardNumber, Expiry, CustomerID)
- **Order**(OrderID, OrderDate, TotalPrice, Status, CustomerID, ShippingAddressID)
- **OrderItem**(OrderItemID, Quantity, UnitPrice, OrderID, ProductID, VendorID)
- **Product**(ProductID, Name, Price, Category, VendorID)
- **Vendor**(VendorID, Name, Email, Phone)
- **Inventory**(InventoryID, ProductID, StockQuantity)
- **Payment**(PaymentID, Method, Amount, Status, OrderID)
- **Shipping**(ShippingID, Carrier, Cost, Status, OrderID)

Relationships and Cardinalities

- Customer **places** Order: 1..* Orders per Customer, each Order by 1 Customer
- Order **contains** Product via OrderItem: 1..* Products per Order, each Product in 0..* Orders
- Vendor **sells** Product: 1..* Products per Vendor, each Product by 1 Vendor
- Product **maintains** Inventory: 1:1
- Order **paid via** Payment: 1:1..*, each Payment for 1 Order
- Order **shipped via** Shipping: 1:1..*, each Shipping linked to 1 Order
- Customer **has** Address: 1..* addresses per Customer, each Address belongs to 1 Customer
- Customer **links** PaymentMethod: 1..* per Customer, each PaymentMethod belongs to 1 Customer

Mermaid ER Diagram Code

```
erDiagram
    CUSTOMER ||--o{ ADDRESS : has
    CUSTOMER ||--o{ PAYMENTMETHOD : links
    CUSTOMER ||--o{ ORDER : places
    ORDER ||--o{ ORDERITEM : contains
    PRODUCT ||--o{ ORDERITEM : included_in
    VENDOR ||--o{ PRODUCT : sells
    PRODUCT ||--|| INVENTORY : maintains
    ORDER ||--o{ PAYMENT : paid_via
    ORDER ||--o{ SHIPPING : shipped_via

    CUSTOMER {
        string CustomerID PK
        string Name
        string Email
        string Phone
    }
    ADDRESS {
        string AddressID PK
        string Street
        string City
        string State
        string Zip
        string CustomerID FK
```

```
}
PAYMENTMETHOD {
    string PaymentID PK
    string Type
    string CardNumber
    string Expiry
    string CustomerID FK
}
ORDER {
    string OrderID PK
    date OrderDate
    float TotalPrice
    string Status
    string CustomerID FK
    string ShippingAddressID FK
}
ORDERITEM {
    string OrderItemID PK
    int Quantity
    float UnitPrice
    string OrderID FK
    string ProductID FK
    string VendorID FK
}
PRODUCT {
    string ProductID PK
    string Name
    float Price
    string Category
    string VendorID FK
}
VENDOR {
    string VendorID PK
    string Name
    string Email
    string Phone
}
INVENTORY {
    string InventoryID PK
    string ProductID FK
    int StockQuantity
}
PAYMENT {
    string PaymentID PK
    string Method
    float Amount
```

```
    string Status
    string OrderID FK
}
SHIPPING {
    string ShippingID PK
    string Carrier
    float Cost
    string Status
    string OrderID FK
}
```

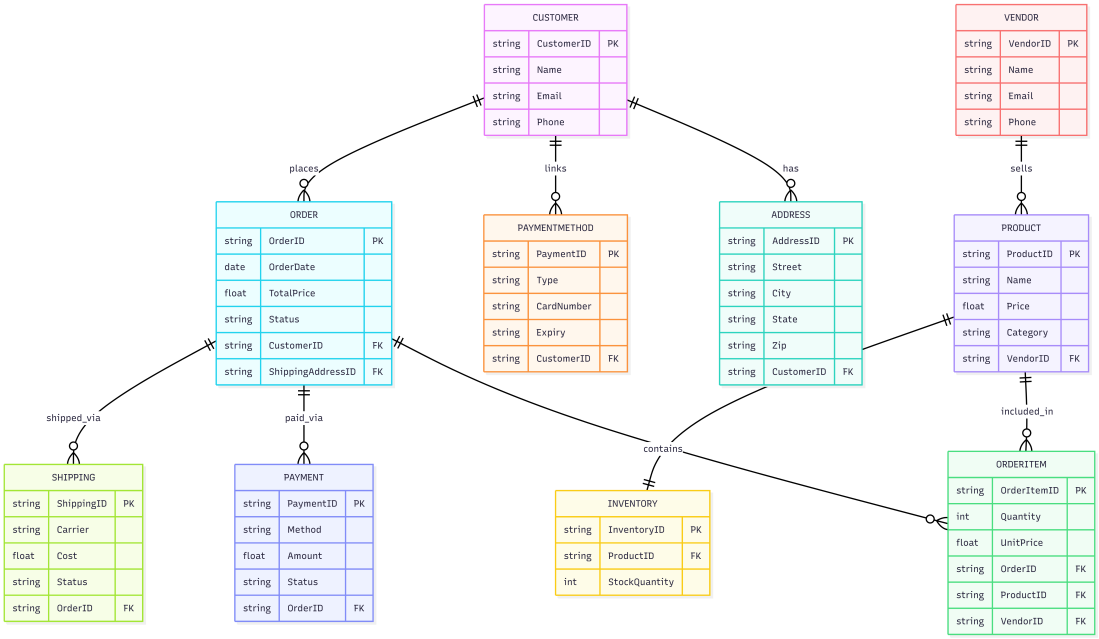

Figure 1: ER Diagram of the E-commerce Platform

Notes:

- All primary keys (PK) are underlined.
- Foreign keys (FK) are indicated in each entity.
- Cardinalities:
  - 1:1, 1..*, 0..* as described in relationships above.